Book 4

Engineering & Implementation Guide

Document Type: Software Engineering Implementation Manual (SEIM)

Version: 1.0

Status: Living Engineering Document

Audience:

- Software Engineers
- ML Engineers
- DevOps Engineers
- Contributors
- Student Developers

Chapter 1

Engineering Principles

Document Information

Property	Value
Status	Approved
Owner	Engineering Team
Priority	Critical
Depends On	Books 1–3

1.1 Purpose This chapter establishes the engineering principles that govern the implementation of Atlas. While Books 1–3 define the product vision, architecture, and evaluation methodology, this book defines how those designs are translated into production-quality software. All contributors are expected to follow the engineering standards documented here.

1.2 Engineering Philosophy Atlas is designed as a long-lived engineering platform rather than a short-term student project. Engineering decisions should prioritize:

- Maintainability
- Readability

- Reproducibility
- Extensibility
- Testability
- Documentation
- Security

Whenever trade-offs arise, long-term maintainability takes precedence over short-term convenience.

1.3 Core Engineering Principles

Principle 1 — Modular Design Every subsystem should have a single, clearly defined responsibility. Examples include:

- Benchmark Engine
- Evaluation Engine
- Reporting Engine
- Execution Adapters
- Authentication Service

Modules communicate through documented interfaces rather than direct internal dependencies.

Principle 2 — Separation of Concerns Business logic must remain independent from:

- User Interface
- Database implementation
- External APIs
- AI providers
- Infrastructure

This allows components to evolve independently.

Principle 3 — Interface-First Development Before implementing a service:

- Define its purpose.
- Define its public interface.
- Define its data contracts.
- Implement internal logic.

This approach reduces coupling and improves maintainability.

Principle 4 — Version Everything Every major artifact in Atlas must be versioned. This includes:

- APIs
- Benchmarks
- Datasets
- Reports
- Database schemas

- Evaluation methodologies
- Configuration files

Versioning ensures backward compatibility and reproducibility.

Principle 5 — Configuration Over Hardcoding Behavior should be controlled through configuration rather than source code. Examples:

- Execution timeouts
- Resource limits
- Supported adapters
- Database connections
- Report templates

Configuration should be externalized using environment variables or configuration files.

Principle 6 — Open by Default Unless restricted by security or licensing requirements:

- APIs should be documented.
- Formats should be open.
- Specifications should be public.
- Interfaces should remain stable.

Atlas aims to encourage community adoption and interoperability.

Principle 7 — Fail Predictably Errors should be:

- Detectable
- Logged
- Recoverable where possible
- Informative

The platform should never fail silently.

Principle 8 — Documentation as Code Engineering documentation evolves alongside the codebase. Every significant architectural decision, API, and subsystem should be documented and version controlled. Documentation is treated as a first-class engineering artifact.

1.4 Engineering Workflow Every feature follows the same lifecycle.

Requirement
↓
Architecture Review
↓
Technical Design
↓
Implementation
↓
Unit Testing

↓
Integration Testing
↓
Code Review
↓
Documentation
↓
Deployment

Skipping stages requires explicit approval.

1.5 Definition of Done A feature is considered complete only when:

- Requirements are satisfied.
- Code is merged.
- Tests pass.
- Documentation is updated.
- APIs are documented.
- Security review is complete (where applicable).
- No critical issues remain.

Code alone does not constitute a completed feature.

1.6 Repository Organization The Atlas repository is organized by feature and responsibility rather than by technology. Examples:

- Backend services
- Frontend applications
- Shared libraries
- Infrastructure
- Documentation
- SDKs
- CLI tools

The detailed repository structure is defined in Chapter 3.

1.7 Code Ownership Every major subsystem has an assigned owner. Responsibilities include:

- Reviewing changes.
- Maintaining documentation.
- Managing technical debt.
- Approving architectural modifications.

Ownership improves accountability and consistency.

1.8 Technical Debt Technical debt should be:

- Documented.
- Prioritized.
- Tracked.
- Reviewed periodically.

Intentional shortcuts must include a justification and a remediation plan.

1.9 Engineering Metrics The engineering team tracks:

- Test coverage
- Build success rate
- Code review turnaround
- Bug resolution time
- Documentation coverage
- Deployment frequency

These metrics measure engineering quality rather than product success.

1.10 Builder Notes

Property	Value
Difficulty	Low
Owner	Engineering Team
Prototype Required	No
ML Required	No

Engineering Notes

Confirmed Decisions

- Atlas follows modular engineering principles.
- Interfaces are defined before implementation.
- Documentation is a first-class artifact.
- All major components are versioned.
- Configuration is externalized.

Open Questions

- Which code formatting tools should become mandatory?
- How should architectural reviews be conducted for community contributions?
- What engineering metrics should be displayed on internal dashboards?

Chapter 2

Technology Stack & Engineering Decisions

Document Type: Engineering Technology Specification (ETS) **Version:** ETS v1.0

Document Information

Property	Value
Status	Approved
Owner	Core Engineering Team
Priority	Critical
Depends On	Books 1–3

2.1 Purpose The technology stack defines the engineering foundation of Atlas. Every technology selected for Version 1 has been evaluated according to:

- Engineering maturity
- Long-term maintainability
- Community support
- Learning curve
- Open-source ecosystem
- Performance
- Cost
- Student team feasibility

The Version 1 stack intentionally favors stability and developer productivity over experimental technologies.

2.2 Technology Selection Principles Every technology should satisfy at least five of the following criteria:

- Open source where practical
- Cross-platform
- Well documented
- Large community support
- Long-term maintenance
- Production ready
- Easy onboarding
- Active ecosystem
- Compatible with Docker

2.3 High-Level Technology Stack

Frontend
    |
Next.js + React + TypeScript
    |
————————————————
Backend
    |
FastAPI + Python
    |
————————————————
Database
    |
PostgreSQL
    |
————————————————
Cache
    |
Redis
    |
————————————————
Object Storage
    |
MinIO (Local) / S3 Compatible
    |
————————————————
Execution
    |
Docker
    |
————————————————
AI Runtime
    |
Ollama

2.4 Backend Stack Language Python 3.12+ Why?

- Excellent AI ecosystem.
- Fast development.
- Rich scientific libraries.
- Strong community.

Alternatives considered:

- Go

- Java
- Rust
- Node.js

Decision: Python offers the highest productivity for an AI evaluation platform.

Framework FastAPI Reasons:

- Native async support.
- Automatic OpenAPI generation.
- Type validation.
- High performance.
- Excellent documentation.

Alternatives:

- Flask
- Django
- Express
- Spring Boot

Decision: FastAPI provides the best balance of speed, maintainability, and developer experience.

2.5 Frontend Stack

- Framework: Next.js
- Language: TypeScript
- UI: React
- Styling: Tailwind CSS
- Charts: Recharts
- State Management: Zustand
- Forms: React Hook Form
- Validation: Zod

Reasons:

- Modern React ecosystem.
- Strong TypeScript support.
- Excellent routing.
- Easy deployment.
- Large community.

2.6 Database Stack Primary Database PostgreSQL Reasons:

- ACID compliance.
- JSON support.
- Mature ecosystem.
- Excellent indexing.

- Open source.

ORM SQLAlchemy + Alembic Reasons:

- Stable.
- Mature migrations.
- Type-safe models.
- Excellent FastAPI integration.

2.7 Cache Redis Purpose:

- Job queue.
- Session storage.
- Rate limiting.
- Evaluation caching.
- Temporary execution state.

Redis is not the source of truth. PostgreSQL remains authoritative.

2.8 Object Storage Version 1: MinIO Future: Amazon S3 Purpose:

- Benchmark packages.
- Reports.
- Datasets.
- Execution logs.
- Generated artifacts.

Using S3-compatible storage avoids vendor lock-in.

2.9 AI Runtime Version 1: Ollama Reasons:

- Local execution.
- Zero API cost.
- Easy installation.
- Offline support.
- Open-weight models.

Supported models depend on user configuration. Atlas itself remains model-agnostic.

2.10 Container Runtime Docker Purpose:

- Benchmark execution.
- Environment isolation.
- Reproducibility.
- Dependency management.

Every benchmark executes inside a controlled container.

2.11 Authentication JWT Password Hashing: Argon2 Reasons:

- Industry standard.

- Stateless authentication.
- Secure password storage.

OAuth providers may be added later without redesign.

2.12 API Design

- Architecture: REST
- Specification: OpenAPI 3.1
- Serialization: JSON
- Versioning: /api/v1

Reasons:

- Simplicity.
- Tooling.
- Wide adoption.

2.13 Background Jobs Version 1: Celery + Redis Responsibilities:

- Benchmark execution.
- Report generation.
- File processing.
- Cleanup jobs.

2.14 Logging Python Logging: Structured JSON logs. Log Levels:

- INFO
- WARNING
- ERROR
- CRITICAL

Logs remain separate from evaluation results.

2.15 Monitoring Version 1: Simple health endpoints. Future:

- Prometheus
- Grafana

These remain optional until deployment complexity increases.

2.16 Development Tools Mandatory:

- Git
- Docker Desktop
- VS Code
- Ruff
- Black
- MyPy
- Pytest
- Pre-commit

These tools standardize development across contributors.

2.17 Dependency Management

- Python: Poetry
- Frontend: pnpm

Reasons:

- Deterministic builds.
- Lockfiles.
- Faster installs.
- Dependency isolation.

2.18 Technology Decision Matrix

Component	Technology
Backend	FastAPI
Language	Python
Frontend	Next.js
UI	React
Styling	Tailwind
Database	PostgreSQL
ORM	SQLAlchemy
Cache	Redis
Storage	MinIO
AI Runtime	Ollama
Queue	Celery
Containers	Docker

2.19 Builder Notes

Property	Value

Difficulty	Medium
Prototype Required	No
ML Required	No

Engineering Notes

Confirmed Decisions

- Python + FastAPI backend.
- Next.js frontend.
- PostgreSQL as the primary datastore.
- Redis for ephemeral state.
- Docker for isolated execution.
- Ollama for local AI inference.

Cross References

- Book 2 – System Architecture
- Book 4 – Repository Structure
- Book 4 – Backend Architecture

CTO Commentary Version 1 deliberately prioritizes a mature, open-source technology stack that two motivated engineering students can realistically build and maintain. Every selected technology has a strong ecosystem, minimizes licensing costs, and aligns with Atlas's goal of reproducible AI evaluation.

Chapter 3

Repository Structure & Monorepo Organization

Document Type: Repository Architecture Specification (RAS) **Version:** RAS v1.0

Document Information

Property	**Value**
Status	Approved
Owner	Platform Engineering Team
Priority	Critical

3.1 Purpose A well-organized repository reduces onboarding time, simplifies maintenance, and

enables independent development of backend, frontend, infrastructure, and shared libraries. Atlas adopts a monorepo architecture to keep all platform components versioned together while preserving clear module boundaries.

3.2 Repository Philosophy The repository is organized by business responsibility, not by programming language. Every top-level directory corresponds to a major platform capability. This ensures that future contributors can understand the project without first understanding the implementation language.

3.3 Repository Overview

```
atlas/
│
├── apps/
├── services/
├── packages/
├── benchmarks/
├── datasets/
├── sdk/
├── cli/
├── infrastructure/
├── docs/
├── scripts/
├── tests/
├── tools/
├── configs/
├── .github/
├── docker/
├── pyproject.toml
├── package.json
├── docker-compose.yml
└── README.md
```

3.4 Applications

```
apps/
│
├── web-dashboard/
├── admin-panel/
├── benchmark-builder/
└── documentation-site/
```

Responsibilities:

- User-facing interfaces
- Administrative tools
- Benchmark authoring
- Documentation

3.5 Backend Services

```
services/
│
├── auth-service/
├── benchmark-service/
├── evaluation-service/
├── execution-service/
├── reporting-service/
├── leaderboard-service/
├── dataset-service/
├── storage-service/
└── notification-service/
```

Each service owns its domain logic and communicates through documented APIs.

3.6 Shared Packages

```
packages/
│
├── shared-types/
├── database/
├── config/
├── logging/
├── validation/
├── auth/
├── metrics/
├── utilities/
└── ui-components/
```

These packages contain reusable code shared across multiple applications and services.

3.7 Benchmarks

```
benchmarks/
│
├── coding/
```

```
├── reasoning/
├── mathematics/
├── planning/
├── tool-use/
├── safety/
├── templates/
└── registry/
```

Each benchmark follows the Atlas Benchmark Schema defined in Book 3.

3.8 Datasets

```
datasets/
│
├── coding/
├── reasoning/
├── math/
├── safety/
├── metadata/
└── licenses/
```

Datasets are version-controlled independently from benchmark definitions.

3.9 SDK

```
sdk/
│
├── python/
├── typescript/
└── examples/
```

The SDK enables external developers to:

- Submit evaluations.
- Create benchmarks.
- Query reports.
- Integrate Atlas into other systems.

3.10 CLI

```
cli/
│
├── atlas/
```

```
├── commands/
├── templates/
└── tests/
```

Example commands:

- atlas benchmark create
- atlas benchmark validate
- atlas evaluate
- atlas report generate
- atlas dataset register

The CLI is the primary developer interface for local workflows.

3.11 Infrastructure

```
infrastructure/
│
├── docker/
├── compose/
├── nginx/
├── monitoring/
└── deployment/
```

Contains infrastructure-as-code and deployment configurations.

3.12 Documentation

```
docs/
│
├── architecture/
├── api/
├── benchmarks/
├── developer-guide/
├── atlas-bible/
└── adr/
```

Engineering documentation is version-controlled alongside the codebase.

3.13 Testing

```
tests/
│
```

```
├── unit/
├── integration/
├── benchmark/
├── e2e/
└── performance/
```

Testing mirrors the platform architecture.

3.14 Naming Conventions

- Directories: kebab-case
- Python packages: snake_case
- Classes: PascalCase
- Functions: snake_case
- API routes: /api/v1/...
- Environment variables: UPPER_SNAKE_CASE

Consistency improves readability and automation.

3.15 Build Order The recommended implementation sequence is:

1. Shared Packages
2. Database Layer
3. Authentication Service
4. Benchmark Service
5. Execution Service
6. Evaluation Service
7. Reporting Service
8. Frontend Applications
9. CLI
10. SDK

Each stage builds upon previously completed components.

3.16 Builder Notes

Property	**Value**
Difficulty	Medium
Owner	Platform Engineering Team
Prototype Required	Yes

Chapter 4

Coding Standards & Software Engineering Practices

Document Type: Software Development Standards (SDS) **Version:** SDS v1.0

Document Information

Property	Value
Status	Approved
Priority	Critical
Owner	Core Engineering Team
Depends On	Chapters 1–3

4.1 Purpose The purpose of this chapter is to establish a consistent engineering standard for every contributor working on Atlas. Consistent code is easier to review, debug, document, and maintain than individually optimized code. Atlas values readability and maintainability over clever implementations.

4.2 Engineering Philosophy Atlas code should be:

- Predictable
- Modular
- Testable
- Typed
- Self-documenting
- Consistent

Every engineer should be able to understand unfamiliar code without requiring the original author.

4.3 Python Standards

- Python Version: Python 3.12+
- Formatting: Black
- Linting: Ruff
- Static Typing: MyPy
- Testing: Pytest

4.4 Type Hinting Every public function must use type hints. Example

```
def evaluate_benchmark(
    benchmark: Benchmark,
    adapter: ExecutionAdapter
) -> EvaluationResult:
```

Avoid using Any unless absolutely necessary.

4.5 File Size Recommended limits

Component	**Maximum**
Function	40 lines
Class	300 lines
File	600 lines

If these limits are exceeded, consider refactoring.

4.6 Naming Conventions

- Variables: benchmark_id
- Functions: run_evaluation()
- Classes: EvaluationEngine
- Constants: MAX_EXECUTION_TIME
- Enums: BenchmarkStatus

4.7 Folder Responsibilities Every folder should own one responsibility. Bad

- utils/

Good

- evaluation/
- benchmark/
- reporting/
- authentication/

4.8 Dependency Rules Allowed API ↓ Service ↓ Repository ↓ Database

Forbidden Repository ↓ API

Dependencies always flow downward.

4.9 Error Handling Every exception should:

- Explain what failed.
- Explain why.

- Suggest recovery where possible.

Avoid

```
except:
    pass
```

Preferred

```
except ValidationError as error:
    logger.error(...)
```

4.10 Logging Never use print()

Instead logger.info() logger.warning() logger.error()

4.11 Comments Comments explain WHY not WHAT Bad i += 1

Good # Retry counter increases after failed evaluation

4.12 Documentation Every module includes

- Purpose
- Author
- Dependencies

Every public function contains docstrings.

4.13 API Standards REST naming Good

- POST /benchmarks
- GET /benchmarks/{id}
- DELETE /benchmarks/{id}

Bad

- /getBenchmarks
- /deleteBenchmark

4.14 Testing Standards Every feature requires Unit Tests ↓ Integration Tests ↓ Regression Tests ↓ Documentation

No feature is complete without tests.

4.15 Pull Request Rules Before merging ✓ Tests pass ✓ Documentation updated ✓ Code reviewed ✓ Formatting verified ✓ Lint passes

4.16 Code Review Checklist Reviewers verify

- Naming
- Architecture
- Performance
- Documentation
- Security
- Tests

4.17 Builder Notes

Property	Value
Difficulty	Low
Owner	Engineering Team

CTO Commentary Good engineering scales better than clever engineering. Atlas prioritizes consistency, documentation, and modularity so that new contributors can understand the codebase quickly and safely extend it.

Chapter 5

Backend Architecture & Service Design

Document Type: Backend Engineering Specification **Version:** BES v1.0

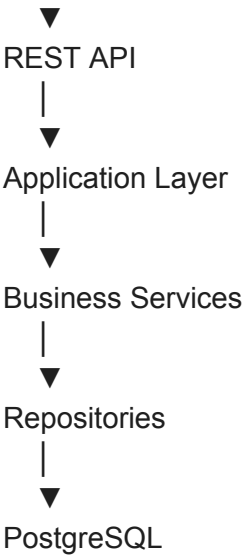
Document Information

Property	Value
Status	Approved
Priority	Critical
Owner	Backend Team

5.1 Purpose The backend is responsible for orchestrating every core operation of Atlas, including authentication, benchmark management, evaluation, reporting, and data persistence. The backend is designed as a modular service-oriented architecture while remaining deployable as a single application during Version 1.

5.2 High-Level Architecture

Client
  |



Infrastructure services such as Redis, Docker, and object storage operate alongside this core flow.

5.3 Layered Architecture The backend is divided into five layers.

Layer	**Responsibility**
API	HTTP endpoints
Service	Business logic
Repository	Data access
Domain	Core models
Infrastructure	External systems

Each layer depends only on the layer immediately below it.

5.4 Core Services Atlas Version 1 contains the following backend services:

- Authentication Service
- User Service
- Project Service
- Benchmark Service
- Dataset Service
- Execution Service
- Evaluation Service

- Reporting Service
- Leaderboard Service
- Storage Service
- Notification Service

Each service owns its own domain logic and public interface.

5.5 Authentication Service Responsibilities:

- User registration
- Login
- JWT generation
- Token validation
- Password hashing
- Session management

Dependencies:

- User Repository
- Security Module

5.6 Benchmark Service Responsibilities:

- Create benchmarks
- Validate schema
- Publish versions
- Archive benchmarks
- Search registry

Dependencies:

- Benchmark Repository
- Dataset Service

5.7 Execution Service Responsibilities:

- Start Docker containers
- Select execution adapter
- Enforce resource limits
- Capture execution logs
- Return raw outputs

This service never performs scoring.

5.8 Evaluation Service Responsibilities:

- Select judge
- Execute evaluation strategy
- Calculate metrics
- Generate capability profile

- Produce evaluation artifacts

This is the heart of Atlas.

5.9 Reporting Service Responsibilities:

- Generate reports
- Export PDF
- Export JSON
- Export CSV
- Build comparison reports

5.10 Leaderboard Service Responsibilities:

- Ranking
- Filtering
- Version-aware comparisons
- Historical snapshots

5.11 Repository Layer Repositories are responsible only for database access. Never place business logic inside repositories. Example BenchmarkRepository ↓ find_by_id() create() update() delete()

5.12 Dependency Injection Every service receives dependencies through constructors. Example

```
EvaluationService(
    benchmark_repository,
    metric_registry,
    execution_service
)
```

Avoid global objects.

5.13 Service Communication Version 1 uses direct service calls.

Benchmark Service ↓ Execution Service ↓ Evaluation Service ↓ Reporting Service

Future message queues are intentionally deferred.

5.14 Error Model Every backend error returns

```
{
  "code": "...",
  "message": "...",
  "details": "...",
  "correlation_id": "..."
}
```

This structure is consistent across all APIs.

5.15 Background Jobs Executed asynchronously:

- Benchmark execution
- Report generation
- File uploads
- Cleanup
- Dataset validation

Celery manages these jobs.

5.16 Builder Notes

Property	Value
Difficulty	High
Owner	Backend Team

Engineering Notes Confirmed Decisions

- Layered architecture.
- Modular services.
- Dependency injection.
- Repository pattern.
- REST APIs.
- Asynchronous background jobs.

Chapter 6

Database Design & Data Persistence

Document Type: Database Architecture Specification (DAS) **Version:** DAS v1.0

Document Information

Property	Value
Status	Approved
Owner	Backend Team
Database	PostgreSQL 16

ORM	SQLAlchemy 2.x
Migration Tool	Alembic

6.1 Purpose The Atlas database stores all persistent platform data. The database is designed around four principles:

- Normalization
- Versioning
- Auditability
- Reproducibility

Atlas stores metadata—not large AI model outputs—inside PostgreSQL. Large artifacts are stored in object storage.

6.2 Database Philosophy Atlas follows:

Business Object
↓
SQLAlchemy Model
↓
Repository
↓
PostgreSQL

The database never contains business logic. Business logic belongs in services.

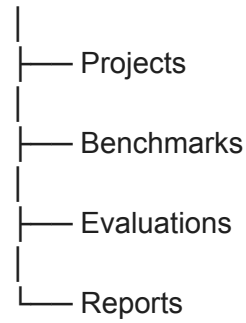
6.3 Database Domains Atlas is divided into logical domains.

- Users
- Projects
- Benchmarks
- Datasets
- Executions
- Evaluations
- Reports
- Leaderboards
- Audit
- Storage

Each domain owns its own tables.

6.4 Core Entity Relationship

User



6.5 Users users

- id
- email
- username
- password_hash
- role
- created_at
- updated_at

Purpose Identity management.

6.6 Projects Projects group evaluations. projects

- id
- owner_id
- name
- description
- visibility
- created_at

Relationships User ↓ Projects ↓ Benchmarks ↓ Executions

6.7 Benchmarks benchmarks

- id
- name
- slug
- version
- category
- status
- creator_id
- created_at

One benchmark ↓ Many versions ↓ Many evaluations

6.8 Benchmark Versions benchmark_versions

- id
- benchmark_id
- version
- schema_version
- dataset_version
- judge_version
- published_at

Immutable after publication.

6.9 Datasets datasets

- id
- name
- license
- checksum
- source
- version

Referenced by benchmarks. Never duplicated.

6.10 Evaluations evaluations

- id
- benchmark_version
- adapter
- status
- started_at
- finished_at

Stores evaluation metadata. Not raw outputs.

6.11 Metrics metrics

- id
- evaluation_id
- metric_name
- value
- unit
- weight

Every metric references Metric Registry.

6.12 Capability Profiles capability_profiles

- evaluation_id
- coding
- reasoning
- planning
- safety
- tool_use

Future capability domains extend this table.

6.13 Reports reports

- id
- evaluation_id
- storage_path
- report_version
- generated_at

PDF JSON CSV stored externally.

6.14 Audit Log Every important action creates an audit event. audit_logs

- id
- actor
- action
- resource
- timestamp
- correlation_id

6.15 Database Indexes Indexes

- Users: email, username
- Projects: owner_id
- Benchmarks: slug
- Evaluations: benchmark_id
- Reports: evaluation_id

6.16 Migration Strategy Every schema change uses Alembic. Migration rules:

- Forward compatible
- Reversible
- Tested

No manual production schema changes.

6.17 Backup Strategy Daily ↓ Incremental ↓ Weekly Full Backup ↓ Monthly Archive

6.18 Builder Notes Difficulty High

CTO Commentary The Atlas database stores relationships, metadata, and scientific provenance rather than large binary artifacts. Keeping benchmark packages, reports, datasets, and logs in object storage while PostgreSQL stores references keeps the system scalable and simplifies backups.

Chapter 7

Authentication, Authorization & Security

Document Type: Security Engineering Specification **Version:** SES v1.0

Document Information

Property	Value
Status	Approved
Owner	Security Team
Priority	Critical

7.1 Purpose Security protects Atlas users, benchmark assets, evaluation artifacts, and system integrity. Security is implemented as a platform concern rather than an application feature. Every service inherits the same authentication and authorization model.

7.2 Security Principles Atlas follows: Least Privilege ↓ Defense in Depth ↓ Secure by Default ↓ Explicit Permissions ↓ Zero Trust between Components

7.3 Authentication Flow

User
↓
Login
↓
Password Verification
↓
JWT Generation
↓
Access Token
↓
Protected API

7.4 Password Storage Passwords are never stored. Only Argon2 hashes are persisted. Requirements Minimum 12 characters.

7.5 JWT Structure JWT contains

- User ID
- Role
- Expiration
- Session ID

Sensitive information is never embedded.

7.6 User Roles Atlas Version 1

Role	Permissions
Guest	Read public data
User	Create projects
Contributor	Publish benchmarks
Moderator	Review submissions
Administrator	Full access

7.7 Authorization Model Every endpoint specifies Required role ↓ Permission ↓ Resource ownership

Example DELETE /benchmark/{id} ↓ Contributor ↓ Owner Only

7.8 API Security Every protected endpoint requires Authorization Bearer Token ↓ Permission Check ↓ Validation ↓ Execution

7.9 Rate Limiting Protect

- Login
- Evaluation API
- Benchmark Creation
- File Uploads

Redis maintains counters.

7.10 Secrets Management Never store secrets in code. Secrets include

- Database URLs

- JWT keys
- Storage credentials
- API tokens

Environment variables only.

7.11 Docker Security Execution containers

- Non-root
- Read-only filesystem
- CPU limits
- Memory limits
- Network isolation
- Execution timeout

Containers are destroyed after execution.

7.12 File Upload Security Every uploaded file is

- Virus scanned (optional for V1)
- File type validated
- Size checked
- Checksum generated

Unsupported files rejected.

7.13 Logging Security Events Security events include

- Failed login
- Permission denied
- Token expiration
- Invalid JWT
- Suspicious activity

Events stored separately from application logs.

7.14 Security Headers Every API returns

- HTTPS only
- CORS policy
- Content Security Policy
- HSTS
- X-Content-Type-Options

7.15 Incident Response Security incident ↓ Detection ↓ Logging ↓ Notification ↓ Containment ↓ Recovery ↓ Postmortem

7.16 Builder Notes Difficulty Medium

Engineering Notes Confirmed

- JWT Authentication
- Argon2 Password Hashing
- Role-Based Access Control
- Docker Isolation
- Redis Rate Limiting
- Environment-Based Secrets

Chapter 8

Execution Adapter SDK & Plugin Architecture

Document Type: Execution Framework Specification (EFS) **Version:** EFS v1.0

Document Information

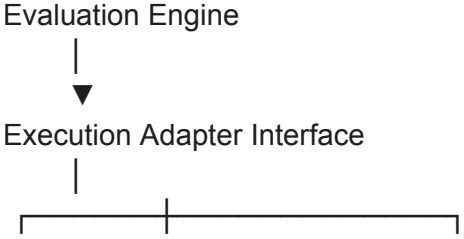
Property	Value
Status	Approved
Owner	Runtime Engineering Team
Priority	Critical
Depends On	Book 2, Book 3

8.1 Purpose Atlas must evaluate models regardless of where they run. Some models execute:

- locally through Ollama,
- through OpenAI-compatible APIs,
- via enterprise-hosted endpoints,
- inside private infrastructure.

The Execution Adapter SDK provides a unified abstraction layer between the Evaluation Engine and model providers. This design keeps Atlas provider-agnostic and prevents coupling to any single AI ecosystem.

8.2 Design Philosophy The Evaluation Engine never communicates directly with AI providers. Instead:



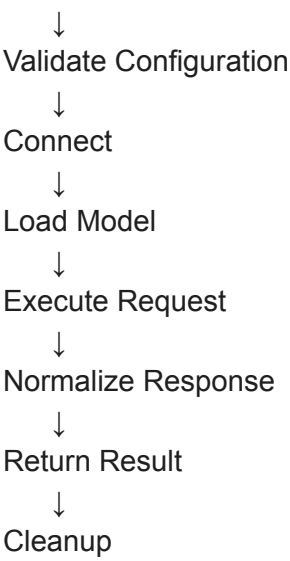
Every execution backend implements the same interface.

8.3 Adapter Responsibilities Every adapter must implement:

- Connection initialization
- Model discovery
- Capability reporting
- Request execution
- Response normalization
- Timeout handling
- Error reporting
- Resource cleanup

Adapters must not contain evaluation logic.

8.4 Adapter Lifecycle



8.5 Adapter Interface Every adapter implements:

```
class ExecutionAdapter:
    def initialize()
    def validate()
    def list_models()
    def execute()
```

```
def health_check()
def shutdown()
```

This interface is stable across all adapters.

8.6 Supported Adapters (Version 1)

Adapter	Status
Ollama Adapter	✅
OpenAI-Compatible API Adapter	✅
Generic REST Adapter	✅
Enterprise Adapter	Experimental

Each adapter is independently versioned.

8.7 Request Object The Evaluation Engine sends a standardized request. Fields include:

- Model identifier
- Benchmark task
- Input prompt
- System prompt
- Temperature
- Max tokens
- Timeout
- Metadata

The adapter translates this request into provider-specific formats.

8.8 Response Object Every adapter returns a normalized response containing:

- Output text
- Completion status
- Runtime
- Token usage (if available)
- Error details
- Raw provider metadata

This abstraction simplifies downstream evaluation.

8.9 Adapter Registry Atlas maintains a registry of installed adapters. Registry metadata:

- Adapter ID

- Version
- Supported protocols
- Supported features
- Health status
- Configuration schema

8.10 Configuration Adapters are configured through YAML files or environment variables. Example:

```
adapter:
  type: ollama
  endpoint: http://localhost:11434
  timeout: 120
```

Configuration is externalized and never hardcoded.

8.11 Error Handling Common adapter errors:

- Connection failure
- Authentication failure
- Model unavailable
- Timeout
- Rate limiting
- Invalid response

Errors are normalized before reaching the Evaluation Engine.

8.12 Health Monitoring Each adapter exposes a health endpoint. Health checks verify:

- Connectivity
- Model availability
- Configuration validity
- Runtime readiness

Failed health checks prevent evaluation execution.

8.13 Plugin Architecture New adapters can be installed as plugins. Each plugin registers:

- Adapter metadata
- Configuration schema
- Supported protocols
- Runtime implementation

Atlas discovers plugins dynamically at startup.

8.14 Builder Notes

Property	Value
Difficulty	High
Prototype Required	Yes

CTO Commentary The Execution Adapter SDK is one of Atlas's most strategic architectural decisions. It isolates provider-specific complexity behind a stable interface, allowing the platform to evolve alongside the rapidly changing AI ecosystem without requiring changes to the Evaluation Engine.

Chapter 9

Evaluation Engine Implementation

Document Type: Runtime Evaluation Specification (RES) **Version:** RES v1.0

Document Information

Property	Value
Status	Approved
Owner	Evaluation Engineering Team
Priority	Critical

9.1 Purpose The Evaluation Engine is the operational core of Atlas. It orchestrates the complete evaluation lifecycle, transforming benchmark definitions and model outputs into capability profiles, reports, and persistent evaluation artifacts. This service is the central coordinator of the platform.

9.2 Responsibilities The Evaluation Engine is responsible for:

- Resolving benchmark definitions
- Selecting execution adapters
- Dispatching evaluation tasks
- Invoking judges
- Calculating metrics
- Generating capability profiles
- Producing reports
- Persisting evaluation artifacts

9.3 Evaluation Pipeline

Receive Request
↓
Validate Request
↓
Resolve Benchmark
↓
Prepare Execution
↓
Dispatch to Adapter
↓
Capture Output
↓
Judge Output
↓
Calculate Metrics
↓
Generate Capability Profile
↓
Generate Report
↓
Persist Results

Each stage is independently logged.

9.4 Internal Components The engine is divided into modular components.

- Request Validator
- Benchmark Resolver
- Adapter Manager
- Task Dispatcher
- Judge Manager
- Metric Calculator
- Capability Mapper
- Report Builder
- Persistence Manager

Each component has a single responsibility.

9.5 Request Validation Validation checks:

- User authorization
- Benchmark availability
- Adapter readiness
- Configuration completeness

- Resource limits

Invalid requests fail before execution begins.

9.6 Benchmark Resolution The engine retrieves:

- Benchmark definition
- Benchmark version
- Dataset references
- Evaluation strategy
- Judge configuration

The resolved benchmark is immutable for the duration of the evaluation.

9.7 Task Dispatch Tasks are dispatched to the selected Execution Adapter. The dispatcher tracks:

- Queue time
- Start time
- Completion time
- Retry attempts
- Timeout status

Each task receives a unique Task ID.

9.8 Judge Invocation After execution, the appropriate judge evaluates the output. Supported judge types:

- Hidden Test Judge
- Exact Match Judge
- Rule-Based Judge
- Metric-Based Judge

Judge outputs are normalized before metric calculation.

9.9 Metric Calculation Metrics are calculated independently. Examples:

- Hidden Test Pass Rate
- Runtime Efficiency
- Compilation Success
- Policy Compliance

Every metric references the Atlas Metric Registry.

9.10 Capability Mapping Metric results are transformed into capability scores using the Atlas Capability Taxonomy. Outputs include:

- Domain Scores
- Capability Vector
- Capability Profile

9.11 Report Generation The Reporting Engine receives:

- Evaluation metadata
- Metrics
- Capability profile
- Evidence chain

It produces:

- PDF report
- JSON report
- CSV metrics export

9.12 Persistence The engine stores:

- Evaluation metadata
- Metric results
- Capability profiles
- Report references
- Audit records

Large artifacts are written to object storage.

9.13 Error Recovery Recoverable failures:

- Temporary adapter outage
- Network interruption
- Queue timeout

Non-recoverable failures:

- Invalid benchmark
- Corrupt dataset
- Unsupported adapter

Recovery strategies are documented and logged.

9.14 Performance Goals Target metrics for Version 1:

Metric	**Target**
Request validation	<100 ms
Benchmark resolution	<200 ms
Adapter dispatch	<100 ms
Report generation	<2 s
API response (excluding evaluation)	<500 ms

Evaluation runtime depends on benchmark complexity.

9.15 Testing Strategy The Evaluation Engine requires:

- Unit tests
- Integration tests
- End-to-end tests
- Performance benchmarks
- Fault injection tests

No engine release proceeds without passing all required test suites.

9.16 Builder Notes

Property	Value
Difficulty	Very High
Prototype Required	Yes

Engineering Notes Confirmed Decisions

- The Evaluation Engine is the orchestration layer.
- Execution, judging, and reporting remain independent components.
- All evaluation stages are logged and auditable.
- Metrics are derived from the Atlas Metric Registry.
- Capability Profiles are generated after metric calculation.

Chapter 10

Reporting Engine Implementation

Document Type: Reporting Engine Engineering Specification (REES) **Version:** REES v1.0

Document Information

Property	Value
Status	Approved
Owner	Reporting Team
Depends On	Book 3, Chapter 10
Priority	Critical

10.1 Purpose The Reporting Engine transforms evaluation artifacts into structured, human-readable, and machine-readable reports. It is responsible for producing professional reports suitable for:

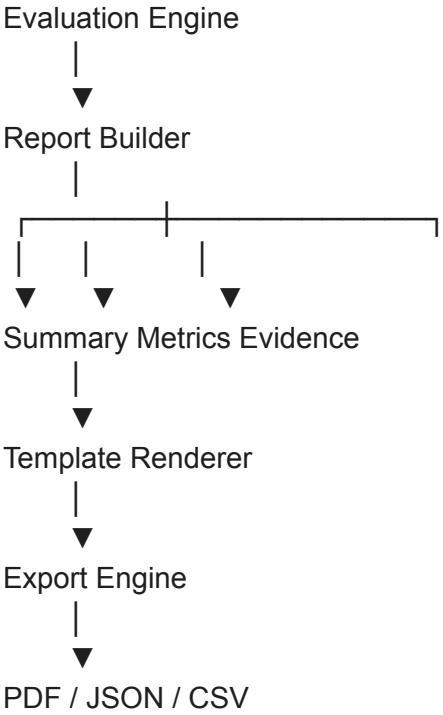
- Researchers
- Students
- Enterprises
- Internal engineering teams

Reports are generated only after evaluation has completed successfully.

10.2 Responsibilities The Reporting Engine is responsible for:

- Collecting evaluation artifacts
- Aggregating metrics
- Building capability summaries
- Rendering charts
- Exporting reports
- Archiving reports

10.3 Internal Architecture



10.4 Core Components The Reporting Engine contains:

- Report Builder

- Template Manager
- Chart Generator
- Export Service
- Archive Manager
- Metadata Builder

Each component performs one clearly defined task.

10.5 Report Builder Responsibilities:

- Load evaluation session
- Retrieve metrics
- Retrieve capability profile
- Assemble report structure

The Report Builder contains no presentation logic.

10.6 Template Manager Templates define report layouts. Version 1 includes:

- Evaluation Summary
- Detailed Evaluation
- Capability Report
- Benchmark Report
- Comparison Report

Templates remain independent of report data.

10.7 Chart Generator Visualizations supported:

- Radar Charts
- Bar Charts
- Capability Heatmaps
- Timeline Graphs
- Benchmark Comparison Tables

Charts are generated dynamically from evaluation data.

10.8 Export Service Supported exports:

- PDF
- JSON
- CSV

Future export formats are intentionally deferred.

10.9 Report Metadata Every report includes:

- Report ID
- Evaluation ID
- Benchmark Version

- Dataset Version
- Capability Taxonomy Version
- Metric Registry Version
- Generation Timestamp

Metadata supports reproducibility.

10.10 Report Storage Reports are stored in object storage. The database stores only:

- Report ID
- Storage Path
- Checksum
- Generation Date

10.11 Report Lifecycle

Evaluation Complete
↓
Build Report
↓
Validate
↓
Export
↓
Archive
↓
Available to User

10.12 Validation Before publication:

- Metadata complete
- Evidence chain present
- Metrics verified
- Capability profile exists
- Export successful

Only validated reports become downloadable.

10.13 Builder Notes

Property	Value
Difficulty	Medium
Prototype Required	Yes

Engineering Notes Confirmed Decisions

- Reporting is independent of evaluation.
- Templates separate presentation from data.
- Reports are reproducible artifacts.
- Object storage stores report files.

CTO Commentary The Reporting Engine transforms raw evaluation outputs into professional, scientifically traceable documents. Its modular architecture allows new report types and export formats to be added without modifying the evaluation pipeline.

Chapter 11

Frontend Architecture & Dashboard Framework

Document Type: Frontend Engineering Specification (FES) **Version:** FES v1.0

Document Information

Property	Value
Status	Approved
Owner	Frontend Engineering Team
Framework	Next.js
Language	TypeScript

11.1 Purpose The Atlas frontend provides the primary interface through which users create benchmarks, execute evaluations, explore results, and manage projects. The frontend emphasizes:

- Simplicity
- Performance
- Accessibility
- Scientific visualization
- Consistency

11.2 Frontend Philosophy The interface should expose complex evaluation data without overwhelming the user. Every screen should answer one primary question. Examples: Dashboard → What is happening? Benchmark Page → What does this benchmark evaluate? Evaluation Page → What happened? Report Page → What do the results mean?

11.3 Application Architecture

Next.js
    |
React Components
    |
Business Hooks
    |
API Client
    |
REST API

Business logic remains in the backend. The frontend focuses on presentation and interaction.

11.4 Route Structure

- /
- dashboard
- projects
- benchmarks
- datasets
- evaluations
- reports
- leaderboards
- settings
- admin

Each route corresponds to a major platform capability.

11.5 Dashboard The dashboard displays:

- Recent evaluations
- Running jobs
- Project summaries
- Capability trends
- Favorite benchmarks
- Notifications

It serves as the operational home page.

11.6 Benchmark Builder The Benchmark Builder provides a guided workflow for creating benchmarks. Features:

- Metadata editor
- Dataset selection
- Capability mapping

- Judge selection
- Validation preview

Drafts can be saved before publication.

11.7 Evaluation Viewer Displays:

- Evaluation status
- Execution timeline
- Metrics
- Capability profile
- Evidence chain
- Logs

The viewer supports real-time updates for active evaluations.

11.8 Report Viewer Allows users to:

- Browse generated reports
- Compare evaluations
- Download exports
- Inspect metadata

Reports are rendered directly from backend-generated data.

11.9 Leaderboards Users can:

- Filter rankings
- Compare evaluations
- Switch capability domains
- View historical snapshots

Every leaderboard displays methodology metadata.

11.10 State Management Global state includes:

- Authentication
- Current project
- User preferences
- Notifications

Evaluation data is fetched on demand rather than stored globally.

11.11 Component Library Reusable components include:

- Tables
- Forms
- Charts
- Cards
- Modals

- Progress Indicators
- Notifications

A consistent design system ensures a unified user experience.

11.12 API Client The frontend communicates exclusively through documented REST APIs. Responsibilities:

- Authentication
- Request handling
- Error normalization
- Retry logic

No business logic is implemented in the API client.

11.13 Accessibility Atlas follows WCAG 2.1 AA guidelines where practical. Requirements:

- Keyboard navigation
- Screen reader compatibility
- Color contrast compliance
- Responsive layouts

11.14 Builder Notes

Property	Value
Difficulty	High
Prototype Required	Yes

Engineering Notes Confirmed Decisions

- Next.js + React frontend.
- Backend owns business logic.
- Dashboard is the operational entry point.
- Component library ensures UI consistency.
- Accessibility is considered from the outset.

Chapter 12

UI/UX Design System & Component Library

Document Type: Design System Specification (DSS) **Version:** DSS v1.0

Document Information

Property	Value

Status	Approved
Owner	Frontend Team
Depends On	Chapter 11
Design Philosophy	Apple × GitHub × Linear × Stripe

12.1 Purpose The Atlas Design System establishes a unified visual language, interaction model, and reusable component library for the entire platform. Rather than designing screens independently, every interface is built from standardized components, design tokens, and interaction patterns. The goal is consistency, accessibility, and scalability.

12.2 Design Philosophy Atlas interfaces should feel:

- Professional
- Scientific
- Minimal
- Data-first
- Calm
- Fast
- Consistent

Visual design must never distract from evaluation data.

12.3 Core Design Principles

Principle 1 — Content First Interfaces prioritize evaluation content over decorative elements.

Principle 2 — Progressive Disclosure Information is revealed in layers. Example:

Overview
↓
Detailed Analysis
↓
Raw Evidence
↓
Execution Logs

Principle 3 — Consistency The same action should behave identically everywhere. Buttons ↓ Forms ↓ Dialogs ↓ Notifications ↓ Tables

Principle 4 — Accessibility Every component supports:

- Keyboard navigation

- Screen readers
- Focus management
- Responsive layouts

12.4 Design Tokens Typography

- Display
- Heading
- Body
- Caption

Spacing

- XS
- SM
- MD
- LG
- XL

Radius

- Small
- Medium
- Large

Elevation

- Card
- Modal
- Dropdown

Color definitions are centralized and referenced through design tokens rather than hardcoded values.

12.5 Layout System Atlas adopts a 12-column responsive grid. Primary layout regions:

Header
↓
Sidebar
↓
Content
↓
Inspector Panel
↓
Footer

The Inspector Panel is used for contextual details without navigating away from the current

page.

12.6 Component Library Core components include:

- Buttons
- Inputs
- Dropdowns
- Tables
- Cards
- Tabs
- Badges
- Tooltips
- Toast Notifications
- Modals
- Progress Indicators
- Breadcrumbs
- Search Bars
- Pagination

Each component has:

- Props
- Accessibility requirements
- Usage guidelines
- Design tokens

12.7 Data Visualization Components Atlas provides reusable visualizations for evaluation data. Components:

- Capability Radar Chart
- Metric Bar Chart
- Benchmark Timeline
- Leaderboard Table
- Capability Heatmap
- Evaluation Timeline
- Evidence Tree
- Benchmark Dependency Graph

All charts consume standardized backend data models.

12.8 Navigation Primary navigation:

- Dashboard
- Projects
- Benchmarks
- Evaluations
- Reports
- Leaderboards

- Datasets
- Settings

Secondary navigation is contextual to the selected page.

12.9 Theme Support Version 1 supports:

- Light Theme
- Dark Theme

Themes share identical spacing, typography, and layout. Only visual tokens change.

12.10 Responsive Design Atlas supports:

- Desktop (primary)
- Tablet
- Laptop
- Large monitors

Mobile support is limited to dashboard viewing in Version 1.

12.11 Accessibility Standards Atlas targets WCAG 2.1 AA compliance. Requirements:

- Contrast ratios
- Keyboard navigation
- Semantic HTML
- ARIA labels
- Screen reader compatibility

12.12 Builder Notes

Property	Value
Difficulty	Medium
Owner	Frontend Team
Prototype Required	Yes

CTO Commentary The Design System ensures that every interface in Atlas feels like part of a single cohesive platform. By standardizing design tokens, components, and layouts, future features can be developed rapidly without sacrificing consistency or accessibility.

Chapter 13

Benchmark Builder & Dataset Management

Document Type: Benchmark Authoring Specification (BAS) **Version:** BAS v1.0

Document Information

Property	Value
Status	Approved
Owner	Benchmark Platform Team
Depends On	Book 3, Chapters 2–9

13.1 Purpose The Benchmark Builder enables users to create, validate, version, publish, and maintain evaluation benchmarks through a structured workflow. It is the primary authoring interface for Atlas.

13.2 Workflow Overview

Create Benchmark
↓
Add Metadata
↓
Attach Dataset
↓
Configure Judge
↓
Map Capabilities
↓
Validate
↓
Publish

Each stage is completed before progressing to the next.

13.3 Benchmark Metadata Required fields:

- Name
- Description
- Category
- Difficulty
- Capability Domains
- License
- Author

- Version

Optional fields:

- Tags
- Documentation
- Examples
- References

13.4 Dataset Management The Dataset Manager allows users to:

- Register datasets
- Import metadata
- Verify licensing
- Track versions
- Associate datasets with benchmarks

Datasets remain independent resources within the Atlas Dataset Registry.

13.5 Validation Engine Before publication, the Validation Engine checks:

- Metadata completeness
- Schema compliance
- Dataset availability
- Judge configuration
- Capability mappings
- Required documentation

Validation errors must be resolved before publishing.

13.6 Version Management Each benchmark version is immutable after publication. Changes require creating a new version. Example:

v1.0
↓
v1.1
↓
v2.0

Historical evaluations always reference the original benchmark version.

13.7 Drafts Users may save incomplete benchmarks as drafts. Drafts support:

- Incremental editing
- Collaboration (future)
- Validation previews

Drafts are not visible in public searches.

13.8 Publication Workflow Publication process:

Draft
↓
Validation
↓
Review
↓
Publish
↓
Registry Update

Only published benchmarks become available for evaluations.

13.9 Benchmark Dashboard Each benchmark includes:

- Metadata
- Version history
- Dataset references
- Capability mappings
- Evaluation statistics
- Documentation
- Change log

This dashboard serves as the canonical source of benchmark information.

13.10 Search & Discovery Users can search benchmarks by:

- Category
- Capability
- Difficulty
- Author
- Version
- License
- Tags

Search results display summary metadata and publication status.

13.11 Benchmark Templates Atlas provides templates for common benchmark categories:

- Coding
- Reasoning
- Mathematics
- Planning
- Tool Use
- Safety

Templates reduce authoring complexity while ensuring consistency.

13.12 Builder Notes

Property	Value
Difficulty	High
Owner	Benchmark Platform Team
Prototype Required	Yes

Engineering Notes Confirmed Decisions

- Benchmarks follow a structured creation workflow.
- Datasets remain independent from benchmarks.
- Validation precedes publication.
- Published benchmark versions are immutable.
- Templates standardize benchmark creation.

Chapter 14

API Specification & SDK Development

Document Type: API Engineering Specification (AES) **Version:** API v1.0

Document Information

Property	Value
Status	Approved
Owner	Platform API Team
Standard	REST + OpenAPI 3.1
Depends On	Chapters 5–13

14.1 Purpose The Atlas API is the primary communication interface between the frontend, SDKs, CLI, and backend services. Every operation performed through the Atlas UI must also be possible through the public API. The API is designed around:

- Simplicity
- Predictability
- Versioning

- Documentation
- Security

14.2 API Design Principles Atlas APIs follow these principles:

- Resource-oriented
- RESTful
- Stateless
- Versioned
- JSON-based
- Idempotent where appropriate

Every endpoint returns structured responses and standardized error objects.

14.3 API Versioning All public APIs use URI versioning. /api/v1/

Future breaking changes require a new major API version.

14.4 Core Resource Groups The API is divided into resource domains:

- Authentication
- Users
- Projects
- Benchmarks
- Datasets
- Evaluations
- Reports
- Leaderboards
- Administration

Each resource owns its own route namespace.

14.5 Endpoint Structure Examples:

```
GET    /api/v1/benchmarks
POST   /api/v1/benchmarks
GET    /api/v1/benchmarks/{id}
PATCH  /api/v1/benchmarks/{id}
DELETE /api/v1/benchmarks/{id}
```

Routes use nouns rather than verbs.

14.6 Request Validation Every request is validated for:

- Authentication
- Authorization
- Input schema
- Required fields

- Business rules

Invalid requests never reach business logic.

14.7 Response Format Successful responses:

```
{
  "success": true,
  "data": {},
  "metadata": {}
}
```

Error responses:

```
{
  "success": false,
  "error": {
    "code": "BENCHMARK_NOT_FOUND",
    "message": "...",
    "correlation_id": "..."
  }
}
```

This format is consistent across the platform.

14.8 Pagination Collection endpoints support:

- page
- limit
- sort
- filter
- search

Responses include pagination metadata.

14.9 API Documentation Atlas automatically generates documentation using OpenAPI. Documentation includes:

- Request examples
- Response examples
- Authentication requirements
- Error codes
- SDK examples

14.10 SDK Strategy Official SDKs:

- Python SDK
- TypeScript SDK

Future SDKs:

- Go
- Java
- Rust

SDKs abstract HTTP communication into strongly typed client libraries.

14.11 CLI Integration The Atlas CLI is implemented using the same public API. This guarantees feature parity between:

- Web Dashboard
- CLI
- SDKs

14.12 API Security Security mechanisms:

- JWT authentication
- Rate limiting
- Input validation
- HTTPS only
- Role-based authorization

14.13 Builder Notes

Property	Value
Difficulty	High
Prototype Required	Yes

CTO Commentary The Atlas API is not merely an implementation detail—it is the public contract of the platform. By ensuring that every feature is accessible through documented APIs, Atlas becomes extensible, scriptable, and easy to integrate into research workflows, enterprise systems, and future automation tools.

Chapter 15

Docker Infrastructure & Secure Execution Sandbox

Document Type: Runtime Infrastructure Specification (RIS) **Version:** RIS v1.0

Document Information

Property	Value
Status	Approved
Owner	Runtime Engineering Team
Depends On	Chapters 5–9

15.1 Purpose Atlas executes untrusted benchmark code and model outputs. To ensure isolation and reproducibility, every execution occurs within a secure Docker sandbox. The sandbox protects:

- Host system
- Evaluation integrity
- User data
- Platform stability

15.2 Design Philosophy Every benchmark execution is isolated.

Evaluation Request
↓
Sandbox Creation
↓
Benchmark Execution
↓
Result Collection
↓
Sandbox Destruction

No execution persists beyond its lifecycle.

15.3 Sandbox Components Each sandbox contains:

- Runtime image
- Benchmark package
- Dataset mount
- Temporary workspace
- Execution logs

The sandbox has no access to unrelated platform resources.

15.4 Security Restrictions Containers run with:

- Non-root user
- Read-only filesystem (where practical)

- CPU limits
- Memory limits
- Process limits
- Network isolation
- Execution timeout

These restrictions minimize the impact of malicious or faulty code.

15.5 Resource Limits Default limits:

Resource	Limit
CPU	Configurable
Memory	Configurable
Disk	Configurable
Execution Time	Configurable

Limits are defined through configuration rather than hardcoded values.

15.6 Benchmark Images Atlas maintains versioned Docker images for supported benchmark environments. Examples:

- Python
- Node.js
- Java
- C/C++
- Rust

Each image is documented and reproducible.

15.7 Filesystem Layout

/workspace
/data
/output
/logs
/tmp

Only /output is persisted after execution.

15.8 Lifecycle Management Container lifecycle:

Create

↓
Configure
↓
Execute
↓
Collect Artifacts
↓
Destroy

Destroyed containers leave no residual state.

15.9 Logging Execution logs include:

- Container ID
- Start time
- End time
- Exit code
- Resource usage
- Errors

Logs become part of the Evaluation Archive.

15.10 Failure Handling Sandbox failures are categorized as:

- Startup failure
- Runtime failure
- Timeout
- Resource exhaustion
- Internal error

Failures are surfaced to the Evaluation Engine through standardized error objects.

15.11 Builder Notes

Property	**Value**
Difficulty	High
Prototype Required	Yes

Engineering Notes Confirmed Decisions

- Docker provides execution isolation.
- Every evaluation runs in a fresh sandbox.
- Containers are ephemeral.

- Resource limits are configurable.
- Logs are archived.